INSTALL KOFFI

You can install Koffi with npm:

```
npm install koffi
```

Once you have installed Koffi, you can start by loading it:

```
// CommonJS syntax

const koffi = require('koffi');



// ES6 modules

import koffi from 'koffi';
```

SIMPLE EXAMPLES

Below you can find two examples:

- The first one runs on Linux. The functions are declared with the C-like prototype language.
- The second one runs on Windows, and uses the node-ffi like syntax to declare functions.

FOR LINUX

This is a small example for Linux systems, which uses `gettimeofday()`, `localtime_r()` and `printf()` to print the current time.

It illustrates the use of output parameters.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



// Load the shared library

const lib = koffi.load('libc.so.6');



// Declare struct types
```

```
const timeval = koffi.struct('timeval', {

    tv_sec: 'unsigned int',

    tv_usec: 'unsigned int'

});

const timezone = koffi.struct('timezone', {

    tz_minuteswest: 'int',

    tz_dsttime: 'int'

});

const time_t = koffi.struct('time_t', { value: 'int64_t' });

const tm = koffi.struct('tm', {

    tm_sec: 'int',

    tm_min: 'int',

    tm_hour: 'int',

    tmay: 'int',

    tm_mon: 'int',

    tm_year: 'int',

    tm_wday: 'int',

    tm_yday: 'int',

    tm_isdst: 'int'

});



// Find functions

const gettimeofday = lib.func('int gettimeofday(_Out_ timeval *tv, _Out_
timezone *tz)');

const localtime_r = lib.func('tm *localtime_r(const time_t *timeval,
_Out_ tm *result)');

const printf = lib.func('int printf(const char *format, ...)');



// Get local time
```

```
let tv = {};

let now = {};

gettimeofday(tv, null);

localtime_r({ value: tv.tv_sec }, now);


// And format it with printf (variadic function)

printf('Hello World!\n');

printf('Local time: %02d:%02d:%02d\n', 'int', now.tm_hour, 'int',
now.tm_min, 'int', now.tm_sec);
```

FOR WINDOWS

This is a small example targeting the Win32 API, using `MessageBox()` to show a *Hello World!* message to the user.

It illustrates the use of the x86 stdcall calling convention, and the use of UTF-8 and UTF-16 string arguments.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


// Load the shared library

const lib = koffi.load('user32.dll');


// Declare constants

const MB_OK = 0x0;

const MB_YESNO = 0x4;

const MB_ICONQUESTION = 0x20;

const MB_ICONINFORMATION = 0x40;

const IDOK = 1;

const IDYES = 6;

const IDNO = 7;
```

```
// Find functions

const MessageBoxA = lib.func('__stdcall', 'MessageBoxA', 'int', ['void
*', 'str', 'str', 'uint']);

const MessageBoxW = lib.func('__stdcall', 'MessageBoxW', 'int', ['void
*', 'str16', 'str16', 'uint']);



let ret = MessageBoxA(null, 'Do you want another message box?', 'Koffi',
MB_YESNO | MB_ICONQUESTION);



if (ret == IDYES)

    MessageBoxW(null, 'Hello World!', 'Koffi', MB_ICONINFORMATION);
```

BUNDLING KOFFI

Please read the dedicated page for information about bundling and packaging applications using Koffi.

BUILD MANUALLY

Follow the build instrutions if you want to build the native Koffi code yourself.

This is only needed if you want to hack on Koffi. The official NPM package provide prebuilt binaries and you don't need to compile anything if you only want to use Koffi in Node.js.

LOADING LIBRARIES

To declare functions, start by loading the shared library with `koffi.load(filename)`.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('/path/to/shared/library'); // File extension
depends on platforms: .so, .dll, .dylib, etc.
```

This library will be automatically unloaded once all references to it are gone (including all the functions that use it, as described below).

Starting with *Koffi 2.3.20*, you can explicitly unload a library by calling `lib.unload()`. Any attempt to find or call a function from this library after unloading it will crash.

On some platforms (such as with the musl C library on Linux), shared libraries cannot be unloaded, so the library will remain loaded and memory mapped after the call to `lib.unload()`.

LOADING OPTIONS

New in Koffi 2.6, changed in Koffi 2.8.2 and Koffi 2.8.6

The `load` function can take an optional object argument, with the following options:

```
const options = {

    lazy: true, // Use RTLD_LAZY (lazy-binding) on POSIX platforms (by
default, use RTLD_NOW)

    global: true, // Use RTLD_GLOBAL on POSIX platforms (by default, use
RTLD_LOCAL)

    deep: true // Use RTLD_DEEPBIND if supported (Linux, FreeBSD)

};



const lib = koffi.load('/path/to/shared/library.so', options);
```

More options may be added if needed.

FUNCTION DEFINITIONS

DEFINITION SYNTAX

Use the object returned by `koffi.load()` to load C functions from the library. To do so, you can use two syntaxes:

- The classic syntax, inspired by node-ffi
- C-like prototypes

CLASSIC SYNTAX

To declare a function, you need to specify its non-mangled name, its return type, and its parameters. Use an ellipsis as the last parameter for variadic functions.

```
const printf = lib.func('printf', 'int', ['str', '...']);
```

```
const atoi = lib.func('atoi', 'int', ['str']);
```

Koffi automatically tries mangled names for non-standard x86 calling conventions. See the section on calling conventions for more information on this subject.

C-LIKE PROTOTYPES

If you prefer, you can declare functions using simple C-like prototype strings, as shown below:

```
const printf = lib.func('int printf(const char *fmt, ...)');

const atoi = lib.func('int atoi(str)'); // The parameter name is not
used by Koffi, and optional
```

You can use `()` or `(void)` for functions that take no argument.

VARIADIC FUNCTIONS

Variadic functions are declared with an ellipsis as the last argument.

In order to call a variadic function, you must provide two Javascript arguments for each additional C parameter, the first one is the expected type and the second one is the value.

```
const printf = lib.func('printf', 'int', ['str', '...']);



// The variadic arguments are: 6 (int), 8.5 (double), 'THE END' (const
char *)

printf('Integer %d, double %g, str %s', 'int', 6, 'double', 8.5, 'str',
'THE END');
```

On x86 platforms, only the Cdecl convention can be used for variadic functions.

CALLING CONVENTIONS

Changed in Koffi 2.7

By default, calling a C function happens synchronously.

Most architectures only support one procedure call standard per process. The 32-bit x86 platform is an exception to this, and Koffi supports several x86 conventions:

Cdecl	`koffi.func(name, ret, params)`	*(default)*	This is the default convention, and the only one on other platforms

Stdcall	`koffi.func('__stdcall', name, ret, params)`	__stdcall	This convention is used extensively within the Win32 API
Fastcall	`koffi.func('__fastcall', name, ret, params)`	__fastcall	Rarely used, uses ECX and EDX for first two parameters
Thiscall	`koffi.func('__thiscall', name, ret, params)`	__thiscall	Rarely used, uses ECX for first parameter

You can safely use these on non-x86 platforms, they are simply ignored.

Support for specifying the convention as the first argument of the classic form was introduced in Koffi 2.7.

In earlier versions, you had to use `koffi.stdcall()` and similar functions. These functions are still supported but deprecated, and will be removed in Koffi 3.0.

Below you can find a small example showing how to use a non-default calling convention, with the two syntaxes:

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('user32.dll');



// The following two declarations are equivalent, and use stdcall on x86
(and the default ABI on other platforms)

const MessageBoxA_1 = lib.func('__stdcall', 'MessageBoxA', 'int', ['void
*', 'str', 'str', 'uint']);

const MessageBoxA_2 = lib.func('int __stdcall MessageBoxA(void *hwnd,
str text, str caption, uint type)');
```

CALL TYPES

SYNCHRONOUS CALLS

Once a native function has been declared, you can simply call it as you would any other JS function.

```
const atoi = lib.func('int atoi(const char *str)');
```

```
let value = atoi('1257');

console.log(value);
```

For variadic functions, you msut specificy the type and the value for each additional argument.

```
const printf = lib.func('printf', 'int', ['str', '...']);



// The variadic arguments are: 6 (int), 8.5 (double), 'THE END' (const
char *)

printf('Integer %d, double %g, str %s', 'int', 6, 'double', 8.5, 'str',
'THE END');
```

ASYNCHRONOUS CALLS

You can issue asynchronous calls by calling the function through its async member. In this case, you need to provide a callback function as the last argument, with `(err, res)` parameters.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('libc.so.6');



const atoi = lib.func('int atoi(const char *str)');



atoi.async('1257', (err, res) => {

    console.log('Result:', res);

})

console.log('Hello World!');



// This program will print:

//   Hello World!

//   Result: 1257
```

These calls are executed by worker threads. It is **your responsibility to deal with data sharing issues** in the native code that may be caused by multi-threading.

You can easily convert this callback-style async function to a promise-based version with `util.promisify()` from the Node.js standard library.

Variadic functions cannot be called asynchronously.

Asynchronous functions run on worker threads. You need to deal with thread safety issues if you share data between threads.

Callbacks must be called from the main thread, or more precisely from the same thread as the V8 intepreter. Calling a callback from another thread is undefined behavior, and will likely lead to a crash or a big mess. You've been warned!

FUNCTION POINTERS

New in Koffi 2.4

You can call a function pointer in two ways:

- Directly call the function pointer with `koffi.call(ptr, type, ...)`
- Decode the function pointer to an actual function with `koffi.decode(ptr, type)`

The example below shows how to call an `int (*)(int, int)` C function pointer both ways, based on the following native C library:

```
typedef int BinaryIntFunc(int a, int b);


static int AddInt(int a, int b) { return a + b; }
static int SubstractInt(int a, int b) { return a - b; }


BinaryIntFunc *GetBinaryIntFunction(const char *type)
{
    if (!strcmp(type, "add")) {
        return AddInt;
    } else if (!strcmp(type, "substract")) {
        return SubstractInt;
    } else {
```

```
        return NULL;

    }

}
```

CALL POINTER DIRECTLY

Use `koffi.call(ptr, type, ...)` to call a function pointer. The first two arguments are the pointer itself and the type of the function you are trying to call (declared with `koffi.proto()` as shown below), and the remaining arguments are used for the call.

```
// Declare function type

const BinaryIntFunc = koffi.proto('int BinaryIntFunc(int a, int b)');



const GetBinaryIntFunction = lib.func('BinaryIntFunc
*GetBinaryIntFunction(const char *name)');



const add_ptr = GetBinaryIntFunction('add');

const substract_ptr = GetBinaryIntFunction('substract');



let sum = koffi.call(add_ptr, BinaryIntFunc, 4, 5);

let delta = koffi.call(substract_ptr, BinaryIntFunc, 100, 58);



console.log(sum, delta); // Prints 9 and 42
```

DECODE POINTER TO FUNCTION

Use `koffi.decode(ptr, type)` to get back a JS function, which you can then use like any other Koffi function.

This method also allows you to perform an asynchronous call with the async member of the decoded function.

```
// Declare function type

const BinaryIntFunc = koffi.proto('int BinaryIntFunc(int a, int b)');
```

```
const GetBinaryIntFunction = lib.func('BinaryIntFunc
*GetBinaryIntFunction(const char *name)');



const add = koffi.decode(GetBinaryIntFunction('add'), BinaryIntFunc);

const substract = koffi.decode(GetBinaryIntFunction('substract'),
BinaryIntFunc);



let sum = add(4, 5);

let delta = substract(100, 58);



console.log(sum, delta); // Prints 9 and 42
```

CONVERSION OF PARAMETERS

By default, Koffi will only forward and translate arguments from Javascript to C. However, many C functions use pointer arguments for output values, or input/output values.

Among other thing, in the the following pages you will learn more about:

- How Koffi translates input parameters to C
- How you can define and use pointers
- How to deal with output parameters

PRIMITIVE TYPES

STANDARD TYPES

While the C standard allows for variation in the size of most integer types, Koffi enforces the same definition for most primitive types, listed below:

void	Undefined	0		Only valid as a return type
int8, int8_t	Number (integer)	1	Signed	
uint8, uint8_t	Number (integer)	1	Unsigned	

char	Number (integer)	1	Signed	
uchar, unsigned char	Number (integer)	1	Unsigned	
char16, char16_t	Number (integer)	2	Signed	
int16, int16_t	Number (integer)	2	Signed	
uint16, uint16_t	Number (integer)	2	Unsigned	
short	Number (integer)	2	Signed	
ushort, unsigned short	Number (integer)	2	Unsigned	
char32, char32_t	Number (integer)	4	Signed	
int32, int32_t	Number (integer)	4	Signed	
uint32, uint32_t	Number (integer)	4	Unsigned	
int	Number (integer)	4	Signed	
uint, unsigned int	Number (integer)	4	Unsigned	
int64, int64_t	Number (integer)	8	Signed	
uint64, uint64_t	Number (integer)	8	Unsigned	
longlong, long long	Number (integer)	8	Signed	
ulonglong, unsigned long long	Number (integer)	8	Unsigned	

float32	Number (float)	4		
float64	Number (float)	8		
float	Number (float)	4		
double	Number (float)	8		

Koffi also accepts BigInt values when converting from JS to C integers. If the value exceeds the range of the C type, Koffi will convert the number to an undefined value. In the reverse direction, BigInt values are automatically used when needed for big 64-bit integers.

Koffi defines a few more types that can change size depending on the OS and the architecture:

bool	Boolean		Usually one byte
long	Number (integer)	Signed	4 or 8 bytes depending on platform (LP64, LLP64)
ulong	Number (integer)	Unsigned	4 or 8 bytes depending on platform (LP64, LLP64)
unsigned long	Number (integer)	Unsigned	4 or 8 bytes depending on platform (LP64, LLP64)
intptr	Number (integer)	Signed	4 or 8 bytes depending on register width
intptr_t	Number (integer)	Signed	4 or 8 bytes depending on register width
uintptr	Number (integer)	Unsigned	4 or 8 bytes depending on register width
uintptr_t	Number (integer)	Unsigned	4 or 8 bytes depending on register width
wchar_t	Number (integer)	*Undefined*	2 bytes on Windows, 4 bytes Linux, macOS and BSD
str, string	String		JS strings are converted to and from UTF-8

str16, string16	String		JS strings are converted to and from UTF-16 (LE)
str32, string32	String		JS strings are converted to and from UTF-32 (LE)

Primitive types can be specified by name (in a string) or through `koffi.types`:

```
// These two lines do the same:

let struct1 = koffi.struct({ dummy: 'long' });

let struct2 = koffi.struct({ dummy: koffi.types.long });
```

ENDIAN-SENSITIVE INTEGERS

New in Koffi 2.1

Koffi defines a bunch of endian-sensitive types, which can be used when dealing with binary data (network payloads, binary file formats, etc.).

int16_le, int16_le_t	2	Signed	Little Endian
int16_be, int16_be_t	2	Signed	Big Endian
uint16_le, uint16_le_t	2	Unsigned	Little Endian
uint16_be, uint16_be_t	2	Unsigned	Big Endian
int32_le, int32_le_t	4	Signed	Little Endian
int32_be, int32_be_t	4	Signed	Big Endian
uint32_le, uint32_le_t	4	Unsigned	Little Endian
uint32_be, uint32_be_t	4	Unsigned	Big Endian
int64_le, int64_le_t	8	Signed	Little Endian
int64_be, int64_be_t	8	Signed	Big Endian
uint64_le, uint64_le_t	8	Unsigned	Little Endian

uint64_be, uint64_be_t	8	Unsigned	Big Endian

STRUCT TYPES

STRUCT DEFINITION

Koffi converts JS objects to C structs, and vice-versa.

Unlike function declarations, as of now there is only one way to create a struct type, with the `koffi.struct()` function. This function takes two arguments: the first one is the name of the type, and the second one is an object containing the struct member names and types. You can omit the first argument to declare an anonymous struct.

The following example illustrates how to declare the same struct in C and in JS with Koffi:

```
typedef struct A {
    int a;
    char b;
    const char *c;
    struct {
        double d1;
        double d2;
    } d;
} A;
const A = koffi.struct('A', {
    a: 'int',
    b: 'char',
    c: 'const char *', // Koffi does not care about const, it is ignored
    d: koffi.struct({
        d1: 'double',
```

```
        d2: 'double'

    })

});
```

Koffi automatically follows the platform C ABI regarding alignment and padding. However, you can override these rules if needed with:

- Pack all members without padding with `koffi.pack()` (instead of `koffi.struct()`)
- Change alignment of a specific member as shown below

```
// This struct is 3 bytes long

const PackedStruct = koffi.pack('PackedStruct', {

    a: 'int8_t',

    b: 'int16_t'

});



// This one is 10 bytes long, the second member has an alignment
requirement of 8 bytes

const BigStruct = koffi.struct('BigStruct', {

    a: 'int8_t',

    b: [8, 'int16_t']

})
```

Once a struct is declared, you can use it by name (with a string, like you can do for primitive types) or through the value returned by the call to `koffi.struct()`. Only the latter is possible when declaring an anonymous struct.

```
// The following two function declarations are equivalent, and declare a
function taking an A value and returning A

const Function1 = lib.func('A Function(A value)');

const Function2 = lib.func('Function', A, [A]);
```

OPAQUE TYPES

Many C libraries use some kind of object-oriented API, with a pair of functions dedicated to create and delete objects. An obvious example of this can be found in stdio.h, with the opaque `FILE *` pointer. You can open and close files with `fopen()` and `fclose()`, and manipule the opaque pointer with other functions such as `fread()` or `ftell()`.

In Koffi, you can manage this with opaque types. Declare the opaque type with `koffi.opaque(name)`, and use a pointer to this type either as a return type or some kind of output parameter (with a double pointer).

Opaque types **have changed in version 2.0, and again in version 2.1**.

In Koffi 1.x, opaque handles were defined in a way that made them usable directly as parameter and return types, obscuring the underlying pointer.

Now, you must use them through a pointer, and use an array for output parameters. This is shown in the example below (look for the call to `ConcatNewOut` in the JS part), and is described in the section on output parameters.

In addition to this, you should use `koffi.opaque()` (introduced in Koffi 2.1) instead of `koffi.handle()` which is deprecated, and will be removed eventually in Koffi 3.0.

Consult the migration guide for more information.

The full example below implements an iterative string builder (concatenator) in C, and uses it from Javascript to output a mix of Hello World and FizzBuzz. The builder is hidden behind an opaque type, and is created and destroyed using a pair of C functions: `ConcatNew` (or `ConcatNewOut`) and `ConcatFree`.

```
// Build with: clang -fPIC -o handles.so -shared handles.c -Wall -O2


#include <stdlib.h>
#include <stdbool.h>
#include <stdio.h>
#include <errno.h>
```

```c
#include <string.h>


typedef struct Fragment {
    struct Fragment *next;

    size_t len;
    char str[];
} Fragment;


typedef struct Concat {
    Fragment *first;
    Fragment *last;

    size_t total;
} Concat;


bool ConcatNewOut(Concat **out)
{
    Concat *c = malloc(sizeof(*c));
    if (!c) {
        fprintf(stderr, "Failed to allocate memory: %s\n",
strerror(errno));
        return false;
    }

    c->first = NULL;
    c->last = NULL;
    c->total = 0;
```

```
    *out = c;
    return true;
}

Concat *ConcatNew()
{
    Concat *c = NULL;
    ConcatNewOut(&c);
    return c;
}

void ConcatFree(Concat *c)
{
    if (!c)
        return;

    Fragment *f = c->first;

    while (f) {
        Fragment *next = f->next;
        free(f);
        f = next;
    }

    free(c);
}
```

```
bool ConcatAppend(Concat *c, const char *frag)

{

    size_t len = strlen(frag);



    Fragment *f = malloc(sizeof(*f) + len + 1);

    if (!f) {

        fprintf(stderr, "Failed to allocate memory: %s\n",
strerror(errno));

        return false;

    }



    f->next = NULL;

    if (c->last) {

        c->last->next = f;

    } else {

        c->first = f;

    }

    c->last = f;

    c->total += len;



    f->len = len;

    memcpy(f->str, frag, len);

    f->str[len] = 0;



    return true;

}



const char *ConcatBuild(Concat *c)
```

```
{
    Fragment *r = malloc(sizeof(*r) + c->total + 1);
    if (!r) {
        fprintf(stderr, "Failed to allocate memory: %s\n",
strerror(errno));
        return NULL;
    }

    r->next = NULL;
    r->len = 0;

    Fragment *f = c->first;

    while (f) {
        Fragment *next = f->next;

        memcpy(r->str + r->len, f->str, f->len);
        r->len += f->len;

        free(f);
        f = next;
    }
    r->str[r->len] = 0;

    c->first = r;
    c->last = r;

    return r->str;
```

```
}

// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('./handles.so');


const Concat = koffi.opaque('Concat');

const ConcatNewOut = lib.func('bool ConcatNewOut(_Out_ Concat **out)');

const ConcatNew = lib.func('Concat *ConcatNew()');

const ConcatFree = lib.func('void ConcatFree(Concat *c)');

const ConcatAppend = lib.func('bool ConcatAppend(Concat *c, const char
*frag)');

const ConcatBuild = lib.func('const char *ConcatBuild(Concat *c)');


let c = ConcatNew();

if (!c) {

    // This is stupid, it does the same, but try both versions (return
value, output parameter)

    let ptr = [null];

    if (!ConcatNewOut(ptr))

        throw new Error('Allocation failure');

    c = ptr[0];

}


try {

    if (!ConcatAppend(c, 'Hello... '))

        throw new Error('Allocation failure');

    if (!ConcatAppend(c, 'World!\n'))

        throw new Error('Allocation failure');
```

```
    for (let i = 1; i <= 30; i++) {
        let frag;
        if (i % 15 == 0) {
            frag = 'FizzBuzz';
        } else if (i % 5 == 0) {
            frag = 'Buzz';
        } else if (i % 3 == 0) {
            frag = 'Fizz';
        } else {
            frag = String(i);
        }

        if (!ConcatAppend(c, frag))
            throw new Error('Allocation failure');
        if (!ConcatAppend(c, ' '))
            throw new Error('Allocation failure');
    }

    let str = ConcatBuild(c);
    if (str == null)
        throw new Error('Allocation failure');
    console.log(str);
} finally {
    ConcatFree(c);
}
```

ARRAY TYPES

FIXED-SIZE C ARRAYS

Changed in Koffi 2.7.1

Fixed-size arrays are declared with `koffi.array(type, length)`. Just like in C, they cannot be passed as functions parameters (they degenerate to pointers), or returned by value. You can however embed them in struct types.

Koffi applies the following conversion rules when passing arrays to/from C:

- **JS to C**: Koffi can take a normal Array (e.g. `[1, 2]`) or a TypedArray of the correct type (e.g. `Uint8Array` for an array of `uint8_t` numbers)
- **C to JS** (return value, output parameters, callbacks): Koffi will use a TypedArray if possible. But you can change this behavior when you create the array type with the optional hint argument: `koffi.array('uint8_t', 64, 'Array')`. For non-number types, such as arrays of strings or structs, Koffi creates normal arrays.

See the example below:

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



// Those two structs are exactly the same, only the array conversion
hint is different

const Foo1 = koffi.struct('Foo1', {

    i: 'int',

    a16: koffi.array('int16_t', 2)

});

const Foo2 = koffi.struct('Foo2', {

    i: 'int',

    a16: koffi.array('int16_t', 2, 'Array')

});
```

```
// Uses an hypothetical C function that just returns the struct passed
as a parameter

const ReturnFoo1 = lib.func('Foo1 ReturnFoo(Foo1 p)');

const ReturnFoo2 = lib.func('Foo2 ReturnFoo(Foo2 p)');



console.log(ReturnFoo1({ i: 5, a16: [6, 8] })) // Prints { i: 5, a16:
Int16Array(2) [6, 8] }

console.log(ReturnFoo2({ i: 5, a16: [6, 8] })) // Prints { i: 5, a16:
[6, 8] }
```

You can also declare arrays with the C-like short syntax in type declarations, as shown below:

```
const StructType = koffi.struct('StructType', {

    f8: 'float [8]',

    self4: 'StructType *[4]'

});
```

The short C-like syntax was introduced in Koffi 2.7.1, use `koffi.array()` for older versions.

FIXED-SIZE STRING BUFFERS

Changed in Koffi 2.9.0

Koffi can also convert JS strings to fixed-sized arrays in the following cases:

- **char arrays** are filled with the UTF-8 encoded string, truncated if needed. The buffer is always NUL-terminated.
- **char16 (or char16_t) arrays** are filled with the UTF-16 encoded string, truncated if needed. The buffer is always NUL-terminated.
- **char32 (or char32_t) arrays** are filled with the UTF-32 encoded string, truncated if needed. The buffer is always NUL-terminated.

Support for UTF-32 and wchar_t (wide) strings was introduced in Koffi 2.9.0.

The reverse case is also true, Koffi can convert a C fixed-size buffer to a JS string. This happens by default for char, char16_t and char32_t arrays, but

you can also explicitly ask for this with the `String` array hint (e.g. `koffi.array('char', 8, 'String')`).

FLEXIBLE ARRAYS

Added in Koffi 2.14.0

C structs ending with a flexible array member are often used for variable-sized structs, and are generally paired with dynamic memory allocation. In many cases, the number of elements is described by another struct member.

Use `koffi.array(type, countedBy, maxLen)` to make a flexible array type, for which the array length is determined by the struct member indicated by the `countedBy` parameter. Flexible array types can only be used as the last member of a struct, as shown below:

```
const FlexibleArray = koffi.struct('FlexibleArray', {

    count: 'size_t',

    numbers: koffi.array('int', 'count', 128)

});
```

For various reasons, Koffi requires you to specify an upper bound (`maxLen`) for the number of elements in the flexible array member.

Also, unlike C flexible arrays, the struct size will expand to accomodate the maximum size of the flexible array.

The following example illustrates how to use a flexible array API in C from Koffi.

```
// Build with: clang -fPIC -o flexible.so -shared flexible.c -Wall -O2


#include <stddef.h>


struct FlexibleArray {

    size_t count;
```

```
    int numbers[];
};

void AppendValues(struct FlexibleArray *arr, size_t count, int start,
int step)
{
    for (size_t i = 0; i < count; i++) {
        arr->numbers[arr->count + i] = start + i * step;
    }
    arr->count += count;
}
```

```
// ES6 syntax: import koffi from 'koffi';
const koffi = require('koffi');

const lib = koffi.load('./flexible.so');

const FlexibleArray = koffi.struct('FlexibleArray', {
    count: 'size_t',
    numbers: koffi.array('int', 'count', 256, 'Array')
});

const AppendValues = lib.func('void AppendValues(_Inout_ FlexibleArray
*arr, int count, int start, int step)');

let array = { count: 0, numbers: [] };

AppendValues(array, 5, 1, 1);
console.log(array); // Prints { count: 5, numbers: [1, 2, 3, 4, 5] }
```

```
AppendValues(array, 3, 10, 2);

console.log(array); // Prints { count: 8, numbers: [1, 2, 3, 4, 5, 10,
12, 14] }
```

This is frequently used in the Win32 API, an exemple of this is AllocateAndInitializeSid().

DYNAMIC ARRAYS (POINTERS)

In C, dynamically-sized arrays are usually passed around as pointers. Read more about array pointers in the relevant section.

UNION TYPES

The declaration and use of union types will be explained in a later section, they are only briefly mentioned here if you need them.

HOW POINTERS ARE USED

In C, pointer arguments are used for differenty purposes. It is important to distinguish these use cases because Koffi provides different ways to deal with each of them:

- **Struct pointers**: Use of struct pointers by C libraries fall in two cases: avoid (potentially) expensive copies, and to let the function change struct contents (output or input/output arguments).
- **Opaque pointers**: the library does not expose the contents of the structs, and only provides you with a pointer to it (e.g. `FILE *`). Only the functions provided by the library can do something with this pointer, in Koffi we call this an opaque type. This is usually done for ABI-stability reason, and to prevent library users from messing directly with library internals.
- **Pointers to primitive types**: This is more rare, and generally used for output or input/output arguments. The Win32 API has a lot of these.
- **Arrays**: in C, you dynamically-sized arrays are usually passed to functions with pointers, either NULL-terminated (or any other sentinel value) or with an additional length argument.

POINTER TYPES

STRUCT POINTERS

The following Win32 example uses `GetCursorPos()` (with an output parameter) to retrieve and show the current cursor position.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('user32.dll');



// Type declarations

const POINT = koffi.struct('POINT', {

    x: 'long',

    y: 'long'

});



// Functions declarations

const GetCursorPos = lib.func('int __stdcall GetCursorPos(_Out_ POINT
*pos)');



// Get and show cursor position

let pos = {};

if (!GetCursorPos(pos))

    throw new Error('Failed to get cursor position');

console.log(pos);
```

OPAQUE POINTERS

New in Koffi 2.0

Some C libraries use handles, which behave as pointers to opaque structs. An example of this is the HANDLE type in the Win32 API. If you want to reproduce this behavior, you can define a **named pointer type** to an opaque type, like so:

```
const HANDLE = koffi.pointer('HANDLE', koffi.opaque());



// And now you get to use it this way:

const GetHandleInformation = lib.func('bool __stdcall
GetHandleInformation(HANDLE h, _Out_ uint32_t *flags)');

const CloseHandle = lib.func('bool __stdcall CloseHandle(HANDLE h)');
```

POINTER TO PRIMITIVE TYPES

In javascript, it is not possible to pass a primitive value by reference to another function. This means that you cannot call a function and expect it to modify the value of one of its number or string parameter.

However, arrays and objects (among others) are reference type values. Assigning an array or an object from one variable to another does not invole any copy. Instead, as the following example illustrates, the new variable references the same array as the first:

```
let list1 = [1, 2];

let list2 = list1;



list2[1] = 42;



console.log(list1); // Prints [1, 42]
```

All of this means that C functions that are expected to modify their primitive output values (such as an `int *` parameter) cannot be used directly. However, thanks to Koffi's transparent array support, you can use Javascript arrays to approximate reference semantics with single-element arrays.

Below, you can find an example of an addition function where the result is stored in an `int *` input/output parameter and how to use this function from Koffi.

```
void AddInt(int *dest, int add)

{

    *dest += add;

}
```

You can simply pass a single-element array as the first argument:

```
const AddInt = lib.func('void AddInt(_Inout_ int *dest, int add)');



let sum = [36];

AddInt(sum, 6);



console.log(sum[0]); // Prints 42
```

DYNAMIC ARRAYS

In C, dynamically-sized arrays are usually passed around as pointers. The length is either passed as an additional argument, or inferred from the array content itself, for example with a terminating sentinel value (such as a NULL pointers in the case of an array of strings).

Koffi can translate JS arrays and TypedArrays to pointer arguments. However, because C does not have a proper notion of dynamically-sized arrays (fat pointers), you need to provide the length or the sentinel value yourself depending on the API.

Here is a simple example of a C function taking a NULL-terminated list of strings as input, to calculate the total length of all strings.

```
// Build with: clang -fPIC -o length.so -shared length.c -Wall -O2



#include <stdlib.h>
```

```c
#include <stdint.h>

#include <string.h>


int64_t ComputeTotalLength(const char **strings)

{

    int64_t total = 0;


    for (const char **ptr = strings; *ptr; ptr++) {

        const char *str = *ptr;

        total += strlen(str);

    }


    return total;

}
```

```javascript
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('./length.so');


const ComputeTotalLength = lib.func('int64_t ComputeTotalLength(const
char **strings)');


let strings = ['Get', 'Total', 'Length', null];

let total = ComputeTotalLength(strings);


console.log(total); // Prints 14
```

By default, just like for objects, array arguments are copied from JS to C but not vice-versa. You can however change the direction as documented in the section on output parameters.

HANDLING VOID POINTERS

New in Koffi 2.1

Many C functions use `void *` parameters in order to pass polymorphic objects and arrays, meaning that the data format changes can change depending on one other argument, or on some kind of struct tag member.

Koffi provides two features to deal with this:

- You can use `koffi.as(value, type)` to tell Koffi what kind of type is actually expected, as shown in the example below.
- Buffers and typed JS arrays can be used as values in place everywhere a pointer is expected. See dynamic arrays for more information, for input or output.

The example below shows the use of `koffi.as()` to read the header of a PNG file with `fread()` directly to a JS object.

```
// ES6 syntax: import koffi from 'koffi';
const koffi = require('koffi');

const lib = koffi.load('libc.so.6');

const FILE = koffi.opaque('FILE');

const PngHeader = koffi.pack('PngHeader', {
    signature: koffi.array('uint8_t', 8),
    ihdr: koffi.pack({
        length: 'uint32_be_t',
        chunk: koffi.array('char', 4),
```

```
        width: 'uint32_be_t',
        height: 'uint32_be_t',
        depth: 'uint8_t',
        color: 'uint8_t',
        compression: 'uint8_t',
        filter: 'uint8_t',
        interlace: 'uint8_t',
        crc: 'uint32_be_t'
    })
});


const fopen = lib.func('FILE *fopen(const char *path, const char
*mode)');
const fclose = lib.func('int fclose(FILE *fp)');
const fread = lib.func('size_t fread(_Out_ void *ptr, size_t size,
size_t nmemb, FILE *fp)');


let filename = process.argv[2];
if (filename == null)
    throw new Error('Usage: node png.js <image.png>');


let hdr = {};
{
    let fp = fopen(filename, 'rb');
    if (!fp)
        throw new Error(`Failed to open '${filename}'`);


    try {
```

```
        let len = fread(koffi.as(hdr, 'PngHeader *'), 1,
koffi.sizeof(PngHeader), fp);

        if (len < koffi.sizeof(PngHeader))

            throw new Error('Failed to read PNG header');

    } finally {

        fclose(fp);

    }

}


console.log('PNG header:', hdr);
```

DISPOSABLE TYPES

New in Koffi 2.0

Disposable types allow you to register a function that will automatically called after each C to JS conversion performed by Koffi. This can be used to avoid leaking heap-allocated strings, for example.

Some C functions return heap-allocated values directly or through output parameters. While Koffi automatically converts values from C to JS (to a string or an object), it does not know when something needs to be freed, or how.

For opaque types, such as FILE, this does not matter because you will explicitly call `fclose()` on them. But some values (such as strings) get implicitly converted by Koffi, and you lose access to the original pointer. This creates a leak if the string is heap-allocated.

To avoid this, you can instruct Koffi to call a function on the original pointer once the conversion is done, by creating a **disposable type** with `koffi.dispose(name, type, func)`. This creates a type derived from another type, the only difference being that *func* gets called with the original pointer once the value has been converted and is not needed anymore.

The *name* can be omitted to create an anonymous disposable type. If *func* is omitted or is null, Koffi will use `koffi.free(ptr)` (which calls the standard C library *free* function under the hood).

```
const AnonHeapStr = koffi.disposable('str'); // Anonymous type (cannot
be used in function prototypes)

const NamedHeapStr = koffi.disposable('HeapStr', 'str'); // Same thing,
but named so usable in function prototypes

const ExplicitFree = koffi.disposable('HeapStr16', 'str16', koffi.free);
// You can specify any other JS function
```

The following example illustrates the use of a disposable type derived from *str*.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('libc.so.6');


const HeapStr = koffi.disposable('str');

const strdup = lib.cdecl('strdup', HeapStr, ['str']);


let copy = strdup('Hello!');

console.log(copy); // Prints Hello!
```

When you declare functions with the prototype-like syntax, you can either use named disposable types or use the '!' shortcut qualifier with compatibles types, as shown in the example below. This qualifier creates an anonymous disposable type that calls `koffi.free(ptr)`.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('libc.so.6');
```

```
// You can also use: const strdup = lib.func('const char *! strdup(const
char *str)')

const strdup = lib.func('str! strdup(const char *str)');



let copy = strdup('World!');

console.log(copy); // Prints World!
```

Disposable types can only be created from pointer or string types.

Be careful on Windows: if your shared library uses a different CRT (such as msvcrt), the memory could have been allocated by a different malloc/free implementation or heap, resulting in undefined behavior if you use `koffi.free()`.

EXTERNAL BUFFERS (VIEWS)

New in Koffi 2.11.0

You can access unmanaged memory with `koffi.view(ptr, len)`. This function takes a pointer and a length, and creates an ArrayBuffer through which you can access the underlying memory without copy.

Some runtimes (such as Electron) forbid the use of external buffers. In this case, trying to create a view will trigger an exception.

The following Linux example writes the string "Hello World!" to a file named "hello.txt" through mmaped memory, to demontrate the use of `koffi.view()`:

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const libc = koffi.load('libc.so.6');



const mode_t = koffi.alias('mode_t', 'uint32_t');

const off_t = koffi.alias('off_t', 'int64_t');
```

```
// These values are used on Linux and may differ on other systems

const O_RDONLY = 00000000;

const O_WRONLY = 00000001;

const O_RDWR = 00000002;

const O_CREAT = 00000100;

const O_EXCL = 00000200;

const O_CLOEXEC = 02000000;


// These values are used on Linux and may differ on other systems

const PROT_READ = 0x01;

const PROT_WRITE = 0x02;

const PROT_EXEC = 0x04;

const MAP_SHARED = 0x01;

const MAP_PRIVATE = 0x02;


const open = libc.func('int open(const char *path, int flags, uint32_t
mode)');

const close = libc.func('int close(int fd)');

const ftruncate = libc.func('int ftruncate(int fd, off_t length)');

const mmap = libc.func('void *mmap(void *addr, size_t length, int prot,
int flags, int fd, off_t offset)');

const munmap = libc.func('int munmap(void *addr, size_t length)');

const strerror = libc.func('const char *strerror(int errnum)');


write('hello.txt', 'Hello World!');


function write(filename, str) {

    let fd = -1;

    let ptr = null;
```

```
    // Work with encoded string
    str = Buffer.from(str);

    try {
        fd = open(filename, O_CREAT | O_EXCL | O_RDWR | O_CLOEXEC,
0644);
        if (fd < 0)
            throw new Error(`Failed to create '${filename}': ` +
strerror(koffi.errno()));

        if (ftruncate(fd, str.length) < 0)
            throw new Error(`Failed to resize '${filename}': ` +
strerror(koffi.errno()));

        ptr = mmap(null, str.length, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
        if (ptr == null)
            throw new Error(`Failed to map '${filename}' to memory: ` +
strerror(koffi.errno()));

        let ab = koffi.view(ptr, str.length);
        let view = new Uint8Array(ab);

        str.copy(view);
     } finally {
        if (ptr != null)
            munmap(ptr, str.length);
        close(fd);
    }
```

```
}
```

UNWRAP POINTERS

You can use `koffi.address(ptr)` on a pointer to get the numeric value as a BigInt object.

OUTPUT AND INPUT/OUTPUT

For simplicity, and because Javascript only has value semantics for primitive types, Koffi can marshal out (or in/out) multiple types of parameters:

- Structs (to/from JS objects)
- Unions
- Opaque types
- String buffers

In order to change an argument from input-only to output or input/output, use the following functions:

- `koffi.out()` on a pointer, e.g. `koffi.out(koffi.pointer(timeval))` (where timeval is a struct type)
- `koffi.inout()` for dual input/output parameters

The same can be done when declaring a function with a C-like prototype string, with the MSDN-like type qualifiers:

- `_Out_` for output parameters
- `_Inout_` for dual input/output parameters

The Win32 API provides many functions that take a pointer to an empty struct for output, except that the first member of the struct (often named `cbSize`) must be set to the size of the struct before calling the function. An example of such a function is `GetLastInputInfo()`.

In order to use these functions in Koffi, you must define the parameter as `_Inout_`: the value must be copied in (to provide `cbSize` to the function) and then the filled struct must be copied out to JS.

Look at the Win32 example below for more information.

PRIMITIVE VALUE

This Windows example enumerate all Chrome windows along with their PID and their title. The `GetWindowThreadProcessId()` function illustrates how to get a primitive value from an output argument.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const user32 = koffi.load('user32.dll');



const DWORD = koffi.alias('DWORD', 'uint32_t');

const HANDLE = koffi.pointer('HANDLE', koffi.opaque());

const HWND = koffi.alias('HWND', HANDLE);



const FindWindowEx = user32.func('HWND __stdcall FindWindowExW(HWND
hWndParent, HWND hWndChildAfter, const char16_t *lpszClass, const
char16_t *lpszWindow)');

const GetWindowThreadProcessId = user32.func('DWORD __stdcall
GetWindowThreadProcessId(HWND hWnd, _Out_ DWORD *lpdwProcessId)');

const GetWindowText = user32.func('int __stdcall GetWindowTextA(HWND
hWnd, _Out_ uint8_t *lpString, int nMaxCount)');



for (let hwnd = null;;) {

    hwnd = FindWindowEx(0, hwnd, 'Chrome_WidgetWin_1', null);



    if (!hwnd)

        break;



    // Get PID
```

```
let pid;
{
    let ptr = [null];
    let tid = GetWindowThreadProcessId(hwnd, ptr);

    if (!tid) {
        // Maybe the process ended in-between?
        continue;
    }

    pid = ptr[0];
}

// Get window title
let title;
{
    let buf = Buffer.allocUnsafe(1024);
    let length = GetWindowText(hwnd, buf, buf.length);

    if (!length) {
        // Maybe the process ended in-between?
        continue;
    }

    title = koffi.decode(buf, 'char', length);
}

console.log({ PID: pid, Title: title });
```

```
}
```

STRUCT EXAMPLES

POSIX STRUCT EXAMPLE

This example calls the POSIX function `gettimeofday()`, and uses the prototype-like syntax.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('libc.so.6');


const timeval = koffi.struct('timeval', {

    tv_sec: 'unsigned int',

    tv_usec: 'unsigned int'

});

const timezone = koffi.struct('timezone', {

    tz_minuteswest: 'int',

    tz_dsttime: 'int'

});


// The _Out_ qualifiers instruct Koffi to marshal out the values

const gettimeofday = lib.func('int gettimeofday(_Out_ timeval *tv, _Out_
timezone *tz)');


let tv = {};

gettimeofday(tv, null);


console.log(tv);
```

WIN32 STRUCT EXAMPLE

Many Win32 functions that use struct outputs require you to set a size member (often named `cbSize`). These functions won't work with `_Out_` because the size value must be copied from JS to C, use `_Inout_` in this case.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const user32 = koffi.load('user32.dll');



const LASTINPUTINFO = koffi.struct('LASTINPUTINFO', {

    cbSize: 'uint',

    dwTime: 'uint32'

});

const GetLastInputInfo = user32.func('bool __stdcall
GetLastInputInfo(_Inout_ LASTINPUTINFO *plii)');



let info = { cbSize: koffi.sizeof(LASTINPUTINFO) };

let success = GetLastInputInfo(info);



console.log(success, info);
```

OPAQUE TYPE EXAMPLE

This example opens an in-memory SQLite database, and uses the node-ffi-style function declaration syntax.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('sqlite3.so');
```

```
const sqlite3 = koffi.opaque('sqlite3');


// Use koffi.out() on a double pointer to copy out (from C to JS) after
the call

const sqlite3_open_v2 = lib.func('sqlite3_open_v2', 'int', ['str',
koffi.out(koffi.pointer(sqlite3, 2)), 'int', 'str']);

const sqlite3_close_v2 = lib.func('sqlite3_close_v2', 'int',
[koffi.pointer(sqlite3)]);



const SQLITE_OPEN_READWRITE = 0x2;

const SQLITE_OPEN_CREATE = 0x4;



let out = [null];

if (sqlite3_open_v2(':memory:', out, SQLITE_OPEN_READWRITE |
SQLITE_OPEN_CREATE, null) != 0)

    throw new Error('Failed to open database');

let db = out[0];



sqlite3_close_v2(db);
```

STRING BUFFER EXAMPLE

New in Koffi 2.2

This example calls a C function to concatenate two strings to a pre-allocated string buffer. Since JS strings are immutable, you must pass an array with a single string instead.

```
void ConcatToBuffer(const char *str1, const char *str2, char *out)

{

    size_t len = 0;
```

```
    for (size_t i = 0; str1[i]; i++) {
        out[len++] = str1[i];
    }
    for (size_t i = 0; str2[i]; i++) {
        out[len++] = str2[i];
    }

    out[len] = 0;
}
const ConcatToBuffer = lib.func('void ConcatToBuffer(const char *str1,
const char *str2, _Out_ char *out)');

let str1 = 'Hello ';
let str2 = 'Friends!';

// We need to reserve space for the output buffer! Including the NUL
terminator
// because ConcatToBuffer() expects so, but Koffi can convert back to a
JS string
// without it (if we reserve the right size).
let out = ['\0'.repeat(str1.length + str2.length + 1)];

ConcatToBuffer(str1, str2, out);

console.log(out[0]);
```

OUTPUT BUFFERS

In most cases, you can use buffers and typed arrays to provide output buffers. This works as long as the buffer only gets used while the native C function is being called. See transient pointers below for an example.

It is unsafe to keep the pointer around in the native code, or to change the contents outside of the function call where it is provided.

If you need to provide a pointer that will be kept around, allocate memory with koffi.alloc() instead.

TRANSIENT POINTERS

New in Koffi 2.3

You can use buffers and typed arrays for output (and input/output) pointer parameters. Simply pass the buffer as an argument and the native function will receive a pointer to its contents.

Once the native function returns, you can decode the content with `koffi.decode(value, type)` as in the following example:

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('libc.so.6');


const Vec3 = koffi.struct('Vec3', {

    x: 'float32',

    y: 'float32',

    z: 'float32'

})


const memcpy = lib.func('void *memcpy(_Out_ void *dest, const void *src,
size_t size)');


let vec1 = { x: 1, y: 2, z: 3 };

let vec2 = null;
```

```
// Copy the vector in a convoluted way through memcpy
{
    let src = koffi.as(vec1, 'Vec3 *');
    let dest = Buffer.allocUnsafe(koffi.sizeof(Vec3));

    memcpy(dest, src, koffi.sizeof(Vec3));

    vec2 = koffi.decode(dest, Vec3);
}

// CHange vector1, leaving copy alone
[vec1.x, vec1.y, vec1.z] = [vec1.z, vec1.y, vec1.x];

console.log(vec1); // { x: 3, y: 2, z: 1 }
console.log(vec2); // { x: 1, y: 2, z: 3 }
```

See decoding variables for more information about the decode function.

STABLE POINTERS

New in Koffi 2.8

In some cases, the native code may need to change the output buffer at a later time, maybe during a later call or from another thread.

In this case, it is **not safe to use buffers or typed arrays**!

However, you can use `koffi.alloc(type, len)` to allocate memory and get a pointer that won't move, and can be safely used at any time by the native code. Use koffi.decode() to read data from the pointer when needed.

The example below sets up some memory to be used as an output buffer where a concatenation function appends a string on each call.

```
#include <assert.h>
```

```
#include <stddef.h>


static char *buf_ptr;

static size_t buf_len;

static size_t buf_size;


void reset_buffer(char *buf, size_t size)

{

    assert(size > 1);


    buf_ptr = buf;

    buf_len = 0;

    buf_size = size - 1; // Keep space for trailing NUL


    buf_ptr[0] = 0;

}


void append_str(const char *str)

{

    for (size_t i = 0; str[i] && buf_len < buf_size; i++, buf_len++) {

        buf_ptr[buf_len] = str[i];

    }

    buf_ptr[buf_len] = 0;

}

const reset_buffer = lib.func('void reset_buffer(char *buf, size_t
size)');

const append_str = lib.func('void append_str(const char *str)');
```

```
let output = koffi.alloc('char', 64);

reset_buffer(output, 64);


append_str('Hello');

console.log(koffi.decode(output, 'char', -1)); // Prints Hello


append_str(' World!');

console.log(koffi.decode(output, 'char', -1)); // Prints Hello World!
```

UNION DEFINITION

New in Koffi 2.5

You can declare unions with a syntax similar to structs, but with the `koffi.union()` function. This function takes two arguments: the first one is the name of the type, and the second one is an object containing the union member names and types. You can omit the first argument to declare an anonymous union.

The following example illustrates how to declare the same union in C and in JS with Koffi:

```
typedef union IntOrDouble {

    int64_t i;

    double d;

} IntOrDouble;

const IntOrDouble = koffi.union('IntOrDouble', {

    i: 'int64_t',

    d: 'double'

});
```

INPUT UNIONS

PASSING UNION VALUES TO C

You can instantiate an union object with `koffi.Union(type)`. This will create a special object that contains at most one active member.

Once you have created an instance of your union, you can simply set the member with the dot operator as you would with a basic object. Then, simply pass your union value to the C function you wish.

```
const U = koffi.union('U', { i: 'int', str: 'char *' });

const DoSomething = lib.func('void DoSomething(const char *type, U u)');

const u1 = new koffi.Union('U'); u1.i = 42;
const u2 = new koffi.Union('U'); u2.str = 'Hello!';

DoSomething('int', u1);
DoSomething('string', u2);
```

For simplicity, Koffi also accepts object literals with one property (no more, no less) setting the corresponding union member. The example belows uses this to simplify the code shown above:

```
const U = koffi.union('U', { i: 'int', str: 'char *' });

const DoSomething = lib.func('void DoSomething(const char *type, U u)');

DoSomething('int', { i: 42 });
DoSomething('string', { str: 'Hello!' });
```

WIN32 EXAMPLE

The following example uses the SendInput Win32 API to emit the Win+D shortcut and hide windows (show the desktop).

```
// ES6 syntax: import koffi from 'koffi';
const koffi = require('koffi');

// Win32 type and functions

const user32 = koffi.load('user32.dll');

const INPUT_MOUSE = 0;
const INPUT_KEYBOARD = 1;
const INPUT_HARDWARE = 2;

const KEYEVENTF_KEYUP = 0x2;
const KEYEVENTF_SCANCODE = 0x8;

const VK_LWIN = 0x5B;
const VK_D = 0x44;

const MOUSEINPUT = koffi.struct('MOUSEINPUT', {
    dx: 'long',
    dy: 'long',
    mouseData: 'uint32_t',
    dwFlags: 'uint32_t',
    time: 'uint32_t',
    dwExtraInfo: 'uintptr_t'
});
const KEYBDINPUT = koffi.struct('KEYBDINPUT', {
    wVk: 'uint16_t',
    wScan: 'uint16_t',
```

```
    dwFlags: 'uint32_t',
    time: 'uint32_t',
    dwExtraInfo: 'uintptr_t'
});
const HARDWAREINPUT = koffi.struct('HARDWAREINPUT', {
    uMsg: 'uint32_t',
    wParamL: 'uint16_t',
    wParamH: 'uint16_t'
});


const INPUT = koffi.struct('INPUT', {
    type: 'uint32_t',
    u: koffi.union({
        mi: MOUSEINPUT,
        ki: KEYBDINPUT,
        hi: HARDWAREINPUT
    })
});


const SendInput = user32.func('unsigned int __stdcall SendInput(unsigned
int cInputs, INPUT *pInputs, int cbSize)');


// Show/hide desktop with Win+D shortcut


let events = [
    make_keyboard_event(VK_LWIN, true),
    make_keyboard_event(VK_D, true),
    make_keyboard_event(VK_D, false),
```

```
    make_keyboard_event(VK_LWIN, false)
];

SendInput(events.length, events, koffi.sizeof(INPUT));

// Utility

function make_keyboard_event(vk, down) {
    let event = {
        type: INPUT_KEYBOARD,
        u: {
            ki: {
                wVk: vk,
                wScan: 0,
                dwFlags: down ? 0 : KEYEVENTF_KEYUP,
                time: 0,
                dwExtraInfo: 0
            }
        }
    };

    return event;
}
```

OUTPUT UNIONS

Unlike structs, Koffi does not know which union member is valid, and it cannot decode it automatically. You can however use special `koffi.Union` objects for output parameters, and decode the memory after the call.

To decode an output union pointer parameter, create a placeholder object with `new koffi.Union(type)` and pass the resulting object to the function.

After the call, you can dereference the member value you want on this object and Koffi will decode it at this moment.

The following example illustrates the use of `koffi.Union()` to decode output unions after the call.

```
#include <stdint.h>


typedef union IntOrDouble {

    int64_t i;

    double d;

} IntOrDouble;


void SetUnionInt(int64_t i, IntOrDouble *out)

{

    out->i = i;

}


void SetUnionDouble(double d, IntOrDouble *out)

{

    out->d = d;

}
const IntOrDouble = koffi.union('IntOrDouble', {

    i: 'int64_t',

    d: 'double',

    raw: koffi.array('uint8_t', 8)

});
```

```
const SetUnionInt = lib.func('void SetUnionInt(int64_t i, _Out_
IntOrDouble *out)');

const SetUnionDouble = lib.func('void SetUnionDouble(double d, _Out_
IntOrDouble *out)');



let u1 = new koffi.Union('IntOrDouble');

let u2 = new koffi.Union('IntOrDouble');



SetUnionInt(123, u1);

SetUnionDouble(123, u2);



console.log(u1.i, '---', u1.raw); // Prints 123 --- Uint8Array(8) [123,
0, 0, 0, 0, 0, 0, 0]

console.log(u2.d, '---', u2.raw); // Prints 123 --- Uint8Array(8) [0, 0,
0, 0, 0, 0, 69, 64]
```

VARIABLE DEFINITIONS

New in Koffi 2.6

To find an exported and declare a variable, use `lib.symbol(name, type)`. You need to specify its name and its type.

```
int my_int = 42;

const char *my_string = NULL;

const my_int = lib.symbol('my_int', 'int');

const my_string = lib.symbol('my_string', 'const char *');
```

You cannot directly manipulate these variables, use:

- koffi.decode() to read their value
- koffi.encode() to change their valuèe

DECODE TO JS VALUES

New in Koffi 2.2, changed in Koffi 2.3

Use `koffi.decode()` to decode C pointers, wrapped as external objects or as simple numbers.

Some arguments are optional and this function can be called in several ways:

- `koffi.decode(value, type)`: no offset
- `koffi.decode(value, offset, type)`: explicit offset to add to the pointer before decoding

By default, Koffi expects NUL terminated strings when decoding them. See below if you need to specify the string length.

The following example illustrates how to decode an integer and a C string variable.

```
int my_int = 42;

const char *my_string = "foobar";

const my_int = lib.symbol('my_int', 'int');

const my_string = lib.symbol('my_string', 'const char *');


console.log(koffi.decode(my_int, 'int')) // Prints 42

console.log(koffi.decode(my_string, 'const char *')) // Prints "foobar"
```

There is also an optional ending `length` argument that you can use in two cases:

- Use it to give the number of bytes to decode in non-NUL terminated strings: `koffi.decode(value, 'char *', 5)`
- Decode consecutive values into an array. For example, here is how you can decode an array with 3 float values: `koffi.decode(value, 'float', 3)`. This is equivalent to `koffi.decode(value, koffi.array('float', 3))`.

The example below will decode the symbol `my_string` defined above but only the first three bytes.

```
// Only decode 3 bytes from the C string my_string

console.log(koffi.decode(my_string, 'const char *', 3)) // Prints "foo"
```

In Koffi 2.2 and earlier versions, the length argument is only used to decode strings and is ignored otherwise.

ENCODE TO C MEMORY

New in Koffi 2.6

Use `koffi.encode()` to encode JS values into C symbols or pointers, wrapped as external objects or as simple numbers.

Some arguments are optional and this function can be called in several ways:

- `koffi.encode(ref, type, value)`: no offset
- `koffi.encode(ref, offset, type, value)`: explicit offset to add to the pointer before encoding

We'll reuse the examples shown above and change the variable values with `koffi.encode()`.

```
int my_int = 42;

const char *my_string = NULL;

const my_int = lib.symbol('my_int', 'int');

const my_string = lib.symbol('my_string', 'const char *');


console.log(koffi.decode(my_int, 'int')) // Prints 42

console.log(koffi.decode(my_string, 'const char *')) // Prints null


koffi.encode(my_int, 'int', -1);

koffi.encode(my_string, 'const char *', 'Hello World!');


console.log(koffi.decode(my_int, 'int')) // Prints -1

console.log(koffi.decode(my_string, 'const char *')) // Prints "Hello
World!"
```

When encoding strings (either directly or embedded in arrays or structs), the memory will be bound to the raw pointer value and managed by Koffi. You can assign to the same string again and again without any leak or risk of use-after-free.

There is also an optional ending `length` argument that you can use to encode an array. For example, here is how you can encode an array with 3 float values: `koffi.encode(symbol, 'float', [1, 2, 3], 3)`. This is equivalent to `koffi.encode(symbol, koffi.array('float', 3), [1, 2, 3])`.

The length argument did not work correctly before Koffi 2.6.11.

CALLBACK TYPES

Changed in Koffi 2.7

In order to pass a JS function to a C function expecting a callback, you must first create a callback type with the expected return type and parameters. The syntax is similar to the one used to load functions from a shared library.

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



// With the classic syntax, this callback expects an integer and returns
nothing

const ExampleCallback = koffi.proto('ExampleCallback', 'void', ['int']);



// With the prototype parser, this callback expects a double and float,
and returns the sum as a double

const AddDoubleFloat = koffi.proto('double AddDoubleFloat(double d,
float f)');
```

The function `koffi.proto()` was introduced in Koffi 2.4, it was called `koffi.callback()` in earlier versions.

For alternative calling conventions (such as `stdcall` on Windows x86 32-bit), you can specify as the first argument with the classic syntax, or after the return type in prototype strings, like this:

```
const HANDLE = koffi.pointer('HANDLE', koffi.opaque());

const HWND = koffi.alias('HWND', HANDLE);



// These two declarations work the same, and use the __stdcall
convention on Windows x86

const EnumWindowsProc = koffi.proto('bool __stdcall EnumWindowsProc
(HWND hwnd, long lParam)');

const EnumWindowsProc = koffi.proto('__stdcall', 'EnumWindowsProc',
'bool', ['HWND', 'long']);
```

You have to make sure you **get the calling convention right** (such as specifying __stdcall for a Windows API callback), or your code will crash on Windows 32-bit.

Before Koffi 2.7, it was *impossible to use an alternative callback calling convention with the classic syntax*. Use a prototype string or *upgrade to Koffi 2.7* to solve this limitation.

Once your callback type is declared, you can use a pointer to it in struct definitions, as function parameters and/or return types, or to call/decode function pointers.

Callbacks **have changed in version 2.0**.

In Koffi 1.x, callbacks were defined in a way that made them usable directly as parameter and return types, obscuring the underlying pointer.

Now, you must use them through a pointer: `void CallIt(CallbackType func)` in Koffi 1.x becomes `void CallIt(CallbackType *func)` in version 2.0 and newer.

Consult the migration guide for more information.

TRANSIENT AND REGISTERED CALLBACKS

Koffi only uses predefined static trampolines, and does not need to generate code at runtime, which makes it compatible with platforms with hardened W^X migitations (such as PaX mprotect). However, this imposes some restrictions on the maximum number of callbacks, and their duration.

Thus, Koffi distinguishes two callback modes:

- Transient callbacks can only be called while the C function they are passed to is running, and are invalidated when it returns. If the C function calls the callback later, the behavior is undefined, though Koffi tries to detect such cases. If it does, an exception will be thrown, but this is no guaranteed. However, they are simple to use, and don't require any special handling.
- Registered callbacks can be called at any time, but they must be manually registered and unregistered. A limited number of registered callbacks can exist at the same time.

You need to specify the correct calling convention on x86 platforms, or the behavior is undefined (Node will probably crash).
Only *cdecl* and *stdcall* callbacks are supported.

TRANSIENT CALLBACKS

Use transient callbacks when the native C function only needs to call them while it runs (e.g. qsort, progress callback, `sqlite3_exec`). Here is a small example with the C part and the JS part.

```
#include <string.h>



int TransferToJS(const char *name, int age, int (*cb)(const char *str,
int age))

{

    char buf[64];

    snprintf(buf, sizeof(buf), "Hello %s!", str);

    return cb(buf, age);

}
```

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');


const lib = koffi.load('./callbacks.so'); // Fake path


const TransferCallback = koffi.proto('int TransferCallback(const char
*str, int age)');


const TransferToJS = lib.func('TransferToJS', 'int', ['str', 'int',
koffi.pointer(TransferCallback)]);


let ret = TransferToJS('Niels', 27, (str, age) => {

    console.log(str);

    console.log('Your age is:', age);

    return 42;

});

console.log(ret);


// This example prints:

//   Hello Niels!

//   Your age is: 27

//   42
```

REGISTERED CALLBACKS

New in Koffi 2.0 (explicit this binding in Koffi 2.2)

Use registered callbacks when the function needs to be called at a later time (e.g. log handler, event handler, `fopencookie/funopen`).
Call `koffi.register(func, type)` to register a callback function, with two arguments: the JS function, and the callback type.

When you are done, call `koffi.unregister()` (with the value returned by `koffi.register()`) to release the slot. A maximum of 8192 callbacks can exist at the same time. Failure to do so will leak the slot, and subsequent registrations may fail (with an exception) once all slots are used.

The example below shows how to register and unregister delayed callbacks.

```
static const char *(*g_cb1)(const char *name);

static void (*g_cb2)(const char *str);



void RegisterFunctions(const char *(*cb1)(const char *name), void
(*cb2)(const char *str))

{

    g_cb1 = cb1;

    g_cb2 = cb2;

}



void SayIt(const char *name)

{

    const char *str = g_cb1(name);

    g_cb2(str);

}

// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('./callbacks.so'); // Fake path



const GetCallback = koffi.proto('const char *GetCallback(const char
*name)');

const PrintCallback = koffi.proto('void PrintCallback(const char
*str)');
```

```
const RegisterFunctions = lib.func('void RegisterFunctions(GetCallback
*cb1, PrintCallback *cb2)');

const SayIt = lib.func('void SayIt(const char *name)');



let cb1 = koffi.register(name => 'Hello ' + name + '!',
koffi.pointer(GetCallback));

let cb2 = koffi.register(console.log, 'PrintCallback *');



RegisterFunctions(cb1, cb2);

SayIt('Kyoto'); // Prints Hello Kyoto!



koffi.unregister(cb1);

koffi.unregister(cb2);
```

Starting *with Koffi 2.2*, you can optionally specify the `this` value for the function as the first argument.

```
class ValueStore {

    constructor(value) { this.value = value; }

    get() { return this.value; }

}



let store = new ValueStore(42);



let cb1 = koffi.register(store.get, 'IntCallback *'); // If a C function
calls cb1 it will fail because this will be undefined

let cb2 = koffi.register(store, store.get, 'IntCallback *'); // However
in this case, this will match the store object
```

SPECIAL CONSIDERATIONS

DECODING POINTER ARGUMENTS

New in Koffi 2.2, changed in Koffi 2.3

Koffi does not have enough information to convert callback pointer arguments to an appropriate JS value. In this case, your JS function will receive an opaque *External* object.

You can pass this value through to another C function that expects a pointer of the same type, or you can use koffi.decode() function to decode pointer arguments.

The following examples uses it to sort an array of strings in-place with the standard C function `qsort()`:

```
// ES6 syntax: import koffi from 'koffi';

const koffi = require('koffi');



const lib = koffi.load('libc.so.6');



const SortCallback = koffi.proto('int SortCallback(const void *first,
const void *second)');

const qsort = lib.func('void qsort(_Inout_ void *array, size_t count,
size_t size, SortCallback *cb)');



let array = ['foo', 'bar', '123', 'foobar'];



qsort(koffi.as(array, 'char **'), array.length, koffi.sizeof('void *'),
(ptr1, ptr2) => {

    let str1 = koffi.decode(ptr1, 'char *');

    let str2 = koffi.decode(ptr2, 'char *');



    return str1.localeCompare(str2);

});
```

```
console.log(array); // Prints ['123', 'bar', 'foo', 'foobar']
```

ASYNCHRONOUS CALLBACKS

New in Koffi 2.2.2

JS execution is inherently single-threaded, so JS callbacks must run on the main thread. There are two ways you may want to call a callback function from another thread:

- Call the callback from an asynchronous FFI call (e.g. `waitpid.async`)
- Inside a synchronous FFI call, pass the callback to another thread

In both cases, Koffi will queue the call back to JS to run on the main thread, as soon as the JS event loop has a chance to run (for example when you await a promise).

Be careful, you can easily get into a deadlock situation if you call a callback from a secondary thread and your main thread never lets the JS event loop run (for example, if the main thread waits for the secondary thread to finish something itself).

HANDLING OF EXCEPTIONS

If an exception happens inside the JS callback, the C API will receive 0 or NULL (depending on the return value type).

Handle the exception yourself (with try/catch) if you need to handle exceptions differently.

TYPES

INTROSPECTION

New in Koffi 2.0: `koffi.resolve()`*, new in Koffi 2.2:* `koffi.offsetof()`

The value returned by `introspect()` has **changed in version 2.0 and in version 2.2**.

In Koffi 1.x, it could only be used with struct types and returned the object passed to koffi.struct() with the member names and types.

Starting in Koffi 2.2, each record member is exposed as an object containing the name, the type and the offset within the record.

Consult the migration guide for more information.

Use `koffi.introspect(type)` to get detailed information about a type: name, primitive, size, alignment, members (record types), reference type (array, pointer) and length (array).

```
const FoobarType = koffi.struct('FoobarType', {
    a: 'int',
    b: 'char *',
    c: 'double'
});

console.log(koffi.introspect(FoobarType));

// Expected result on 64-bit machines:
// {
//     name: 'FoobarType',
//     primitive: 'Record',
//     size: 24,
//     alignment: 8,
//     members: {
//         a: { name: 'a', type: [External: 4b28a60], offset: 0 },
//         b: { name: 'b', type: [External: 4b292e0], offset: 8 },
//         c: { name: 'c', type: [External: 4b29260], offset: 16 }
//     }
// }
```

Koffi also exposes a few more utility functions to get a subset of this information:

- `koffi.sizeof(type)` to get the size of a type
- `koffi.alignof(type)` to get the alignment of a type
- `koffi.offsetof(type, member_name)` to get the offset of a record member
- `koffi.resolve(type)` to get the resolved type object from a type string

Just like before, you can refer to primitive types by their name or through `koffi.types`:

```
// These two lines do the same:

console.log(koffi.sizeof('long'));

console.log(koffi.sizeof(koffi.types.long));
```

ALIASES

New in Koffi 2.0

You can alias a type with `koffi.alias(name, type)`. Aliased types are completely equivalent.

CIRCULAR REFERENCES

New in Koffi 2.10.0

In some cases, composite types can point to each other and thus depend on each other. This can also happen when a function takes a pointer to a struct that also contains a function pointer.

To deal with this, you can create an opaque type and redefine it later to a concrete struct or union type, as shown below.

```
const Type1 = koffi.opaque('Type1');


const Type2 = koffi.struct('Type2', {

    ptr: 'Type1 *',
```

```
    i: 'int'
});


// Redefine Type1 to a concrete type
koffi.struct(Type1, {
    ptr: 'Type2 *',
    f: 'float'
});
```

You must use a proper type object when you redefine the type. If you only have the name, use `koffi.resolve()` to get a type object from a type string.

```
const MyType = koffi.opaque('MyType');


// This does not work, you must use the MyType object and not a type string
koffi.struct('MyType', {
     ptr: 'Type2 *',
     f: 'float'
});
```

SETTINGS

MEMORY USAGE

For synchronous/normal calls, Koffi uses two preallocated memory blocks:

- One to construct the C stack and assign registers, subsequently used by the platform-specific assembly code (1 MiB by default)
- One to allocate strings and objects/structs (2 MiB by default)

Unless very big strings or objects (at least more than one page of memory) are used, Koffi does not directly allocate any extra memory during calls or callbacks. However, please note that the JS engine (V8) might.

The size (in bytes) of these preallocated blocks can be changed.
Use `koffi.config()` to get an object with the settings,
and `koffi.config(obj)` to apply new settings.

```
let config = koffi.config();

console.log(config);
```

The same is true for asynchronous calls. When an asynchronous call is made, Koffi will allocate new blocks unless there is an unused (resident) set of blocks still available. Once the asynchronous call is finished, these blocks are freed if there are more than `resident_async_pools` sets of blocks left around.

There cannot be more than `max_async_calls` running at the same time.

DEFAULT SETTINGS

sync_stack_size	1 MiB	Stack size for synchronous calls
sync_heap_size	2 MiB	Heap size for synchronous calls
async_stack_size	256 kiB	Stack size for asynchronous calls
async_heap_size	512 kiB	Heap size for asynchronous calls
resident_async_pools	2	Number of resident pools for asynchronous calls
max_async_calls	64	Maximum number of ongoing asynchronous calls
max_type_size	64 MiB	Maximum size of Koffi types (for arrays and structs)

USAGE STATISTICS

New in Koffi 2.3.2

You can use `koffi.stats()` to get a few statistics related to Koffi.

POSIX ERROR CODES

New in Koffi 2.3.14

You can use `koffi.errno()` to get the current errno value, and `koffi.errno(value)` to change it.

The standard POSIX error codes are available in `koffi.os.errno`, as shown below:

```
const assert = require('assert');


// ES6 syntax: import koffi from 'koffi';
const koffi = require('koffi');


const lib = koffi.load('libc.so.6');


const close = lib.func('int close(int fd)');


close(-1);
assert.equal(koffi.errno(), koffi.os.errno.EBADF);


console.log('close() with invalid FD is POSIX compliant!');
```

RESET INTERNAL STATE

New in Koffi 2.5.19

You can use `koffi.reset()` to clear some Koffi internal state such as:

- Parser type names
- Asynchronous function broker (useful to avoid false positive with `jest --detectOpenHandles`)

This function is mainly intended for test code, when you execute the same code over and over and you need to reuse type names.

Trying to use a function or a type that was initially defined before the reset is undefined behavior and will likely lead to a crash!

BUNDLING

NATIVE MODULES

Simplified in Koffi 2.5.9

Koffi uses native modules to work. The NPM package contains binaries for various platforms and architectures, and the appropriate module is selected at runtime.

Please note that Koffi is meant for Node.js (or Electron) and not for browsers! It it not possible to load native libraries inside a browser!

In theory, the **packagers/bundlers should be able to find all native modules** because they are explictly listed in the Javascript file (as static strings) and package them somehow.

If that is not the case, you can manually arrange to copy the `node_modules/koffi/build/koffi` directory next to your bundled script.

Here is an example that would work:

```
koffi/
    win32_x64/
        koffi.node
    linux_x64/
        koffi.node
    ...
MyBundle.js
```

When running in Electron, Koffi will also try to find the native module in `process.resourcesPath`. For an Electron app you could do something like this

```
locales/
resources/
    koffi/
```

```
        win32_ia32/
            koffi.node
        win32_x64/
            koffi.node
        ...
MyApp.exe
```

INDIRECT LOADER

New in Koffi 2.6.2

Some bundlers (such as vite) don't like when require is used with native modules.

In this case, you can use `require('koffi/indirect')` but you will need to make sure that the native Koffi modules are packaged properly.

PACKAGING EXAMPLES

ELECTRON WITH ELECTRON-BUILDER

Packaging with electron-builder should work as-is.

Take a look at the full working example in the repository.

ELECTRON FORGE

Packaging with Electron Force should work as-is, even when using webpack as configured initially when you run:

```
npm init electron-app@latest my-app -- --template=webpack
```

Take a look at the full working example in the repository.

NW.JS

Packagers such as nw-builder should work as-is.

You can find a full working example in the repository.

NODE.JS AND ESBUILD

You can easily tell esbuild to copy the native files with the copy loader and things should just work. Use something like:

```
esbuild index.js --platform=node --bundle --loader:.node=copy --
outdir=dist/
```

You can find a full working example in the repository.

NODE.JS AND YAO-PKG

Use yao-pkg to make binary packages of your Node.js-based project.

You can find a full working example in the repository.

KOFFI 1.X TO 2.X

The API was changed in 2.x in a few ways, in order to reduce some excessively "magic" behavior and reduce the syntax differences between C and the C-like prototypes.

You may need to change your code if you use:

- Callback functions
- Opaque types
- `koffi.introspect()`

CALLBACK TYPE CHANGES

In Koffi 1.x, callbacks were defined in a way that made them usable directly as parameter and return types, obscuring the underlying pointer. Now, you must use them through a pointer: `void CallIt(CallbackType func)` in Koffi 1.x becomes `void CallIt(CallbackType *func)` in version 2.0 and newer.

Given the following C code:

```
#include <string.h>
```

```
int TransferToJS(const char *name, int age, int (*cb)(const char *str,
int age))

{

    char buf[64];

    snprintf(buf, sizeof(buf), "Hello %s!", str);

    return cb(buf, age);

}
```

The two versions below illustrate the API difference between Koffi 1.x and Koffi 2.x:

```
// Koffi 1.x


const TransferCallback = koffi.proto('int TransferCallback(const char
*str, int age)');


const TransferToJS = lib.func('TransferToJS', 'int', ['str', 'int',
TransferCallback]);

// Equivalent to: const TransferToJS = lib.func('int TransferToJS(str s,
int x, TransferCallback cb)');


let ret = TransferToJS('Niels', 27, (str, age) => {

    console.log(str);

    console.log('Your age is:', age);

    return 42;

});

console.log(ret);

// Koffi 2.x


const TransferCallback = koffi.proto('int TransferCallback(const char
*str, int age)');
```

```
const TransferToJS = lib.func('TransferToJS', 'int', ['str', 'int',
koffi.pointer(TransferCallback)]);

// Equivalent to: const TransferToJS = lib.func('int TransferToJS(str s,
int x, TransferCallback *cb)');



let ret = TransferToJS('Niels', 27, (str, age) => {

    console.log(str);

    console.log('Your age is:', age);

    return 42;

});

console.log(ret);
```

Koffi 1.x only supported transient callbacks, you must use Koffi 2.x for registered callbacks.

The function `koffi.proto()` was introduced in Koffi 2.4, it was called `koffi.callback()` in earlier versions.

OPAQUE TYPE CHANGES

In Koffi 1.x, opaque handles were defined in a way that made them usable directly as parameter and return types, obscuring the underlying pointer. Now, in Koffi 2.0, you must use them through a pointer, and use an array for output parameters.

In addition to that, `koffi.handle()` has been deprecated in Koffi 2.1 and replaced with `koffi.opaque()`. They work the same but new code should use `koffi.opaque()`, the former one will eventually be removed in Koffi 3.0.

For functions that return opaque pointers or pass them by parameter:

```
// Koffi 1.x



const FILE = koffi.handle('FILE');

const fopen = lib.func('fopen', 'FILE', ['str', 'str']);
```

```
const fopen = lib.func('fclose', 'int', ['FILE']);


let fp = fopen('EMPTY', 'wb');
if (!fp)
    throw new Error('Failed to open file');
fclose(fp);
// Koffi 2.1


// If you use Koffi 2.0: const FILE = koffi.handle('FILE');
const FILE = koffi.opaque('FILE');
const fopen = lib.func('fopen', 'FILE *', ['str', 'str']);
const fopen = lib.func('fclose', 'int', ['FILE *']);


let fp = fopen('EMPTY', 'wb');
if (!fp)
    throw new Error('Failed to open file');
fclose(fp);
```

For functions that set opaque handles through output parameters (such as `sqlite3_open_v2`), you must now use a single element array as shown below:

```
// Koffi 1.x


const sqlite3 = koffi.handle('sqlite3');


const sqlite3_open_v2 = lib.func('int sqlite3_open_v2(const char *,
_Out_ sqlite3 *db, int, const char *)');
const sqlite3_close_v2 = lib.func('int sqlite3_close_v2(sqlite3 db)');
```

```
const SQLITE_OPEN_READWRITE = 0x2;

const SQLITE_OPEN_CREATE = 0x4;



let db = {};



if (sqlite3_open_v2(':memory:', db, SQLITE_OPEN_READWRITE |
SQLITE_OPEN_CREATE, null) != 0)

    throw new Error('Failed to open database');



sqlite3_close_v2(db);

// Koffi 2.1



// If you use Koffi 2.0: const sqlite3 = koffi.handle('sqlite3');

const sqlite3 = koffi.opaque('sqlite3');



const sqlite3_open_v2 = lib.func('int sqlite3_open_v2(const char *,
_Out_ sqlite3 **db, int, const char *)');

const sqlite3_close_v2 = lib.func('int sqlite3_close_v2(sqlite3 *db)');



const SQLITE_OPEN_READWRITE = 0x2;

const SQLITE_OPEN_CREATE = 0x4;



let db = null;



let ptr = [null];

if (sqlite3_open_v2(':memory:', ptr, SQLITE_OPEN_READWRITE |
SQLITE_OPEN_CREATE, null) != 0)

    throw new Error('Failed to open database');

db = ptr[0];
```

```
sqlite3_close_v2(db);
```

NEW KOFFI.INTROSPECT()

In Koffi 1.x, `koffi.introspect()` would only work with struct types, and return the object passed to `koffi.struct()` to initialize the type. Now this function works with all types.

You can still get the list of struct members:

```
const StructType = koffi.struct('StructType', { dummy: 'int' });


// Koffi 1.x
let members = koffi.introspect(StructType);


// Koffi 2.x
let members = koffi.introspect(StructType).members;
```